

Salivo Programming Language Guide

A comprehensive reference manual, tutorial, and cheat sheet for building high-performance applications with Salivo.

Key Architectural Highlights: Stream Header Imports (`><`), Zero-Underscore Data Types, Deterministic RAIL Memory (`Drop` trait), and Postfix `?` Error Propagation.

1. Stream Header Imports (`><`)

Salivo uses `><` as the sole import operator in the language. It represents input streams flowing into the application module.

```
module myapp.main;

><salivo.std.core::{out, outln, assertEq, Result, ok, err};
><salivo.std.fs::{open, close, pathJoin};
><salivo.std.collections::{Vec, vecNew, push, get, vecLen};
```

2. Complete Data Types Reference

Category	Data Type	Description	Syntax Example
Primitives	<code>int</code>	64-bit signed integer	<code>let x: int = 42;</code>
	<code>float</code>	64-bit floating point number	<code>let pi: float = 3.14159;</code>
	<code>bool</code>	Boolean truth value	<code>let flag: bool = true;</code>
	<code>string</code>	Owned UTF-8 string	<code>let s: string = "Salivo";</code>
	<code>StrSlice</code>	Borrowed string slice	<code>let slice: StrSlice = "Hello";</code>
Monads	<code>Option<T></code>	Optional value: <code>some(v)</code> or <code>none()</code>	<code>let val: Option<int> = some(5);</code>
	<code>Result<T, E></code>	Success or Error result	<code>let res: Result<int, string> = ok(10);</code>
Collections	<code>Vec<T></code>	Dynamic vector array	<code>let mut v = vecNew<int>();</code>
	<code>Map<K, V></code>	Hash map table	<code>let mut m = mapNew<string, int>();</code>
	<code>Heap<T></code>	Binary min/max heap	<code>let mut h = heapNew<int>();</code>

Category	Data Type	Description	Syntax Example
	<code>BTree<K, V></code>	B-Tree map	<code>let mut t = btreeNew<string, int> (());</code>

3. Hello World Program

```
><salivo.std.core::{outln};  
  
pub func main() -> int {  
    outln("Hello, Salivo!");  
    return 0;  
}
```

4. Input, Output & String Interpolation

```
><salivo.std.core::{out, outln};  
  
pub func main() -> int {  
    out("Enter your name:");  
    let name = input();  
  
    // Formatting with postfix modifiers (:trim and :upper)  
    outln($"Hello, {name:trim:upper}!");  
    return 0;  
}
```

5. Control Flow & Loops

3-Clause C-Style **for** Loop

```
for (let mut i = 0; i < 5; i = i + 1) {  
    outln($"Index: {i:str}");  
}
```

while Loop & Pattern Matching

```
let mut count = 3;  
while (count > 0) {  
    count = count - 1;  
}  
  
match status {  
    0 => "Pending",  
    1 => "Active",  
    _ => "Unknown",  
}
```

6. Functions & Postfix **?** Operator

```
pub func safeDivide(int a, int b) -> Result<int, string> {  
    if (b == 0) { return err<int, string>("Division by zero"); }  
    return ok<int, string>(a / b);  
}  
  
pub func readConfig(string path) -> Result<string, IOError> {  
    let handle = open(path, "r")?; // Auto-unwraps or returns Err  
    return ok<string, IOError>(readStr(path)?);  
}
```

7. Structs, Enums & RAII **Drop**

```
pub struct UserProfile {  
    int id;  
    string username;  
}  
  
pub trait Drop {  
    func drop(self);  
}  
  
impl Drop for UserProfile {  
    pub func drop(self) {  
        outln($"Cleaning up user session {self.id:str}");  
    }  
}
```